

Data Structures Lab Algorithms (SPPU Practical Ready)

Experiment 1 – Linear Search Algorithm

1. Start
2. Read the number of students n
3. Declare array arr[n]
4. Input roll numbers into array
5. Read roll number key to be searched
6. Set variable flag = 0
7. Set i = 0
8. Repeat Steps 9 to 11 while i < n
9. Compare arr[i] with key
10. If arr[i] == key then display "Student attended training program", set flag = 1 and go to Step 13
11. Increment i = i + 1
12. End loop
13. If flag == 0 then display "Student not found"
14. Stop

Experiment 1 – Binary Search Algorithm

1. Start
2. Read number of students n
3. Input sorted roll numbers in array arr[n]
4. Read roll number key
5. Initialize low = 0, high = n - 1, flag = 0
6. Repeat Steps 7 to 13 while low <= high
7. Calculate mid = (low + high) / 2
8. Compare arr[mid] with key
9. If arr[mid] == key then display "Student found", set flag = 1 and go to Step 15
10. Else if key > arr[mid] then set low = mid + 1
11. Else set high = mid - 1
12. Repeat loop
13. End loop
14. If flag == 0 then display "Student not found"
15. Stop

Experiment 2 – Bubble Sort Algorithm

1. Start
2. Read number of students n
3. Input percentages into array a[n]
4. Set i = 0
5. Repeat Steps 6 to 10 while i < n-1
6. Set j = 0
7. Repeat Steps 8 to 9 while j < n-i-1
8. If a[j] > a[j+1] then swap both elements
9. Increment j = j + 1
10. Increment i = i + 1
11. Display sorted percentages
12. Display top five percentages

13. Stop

Experiment 2 – Selection Sort Algorithm

1. Start
2. Read number of students n
3. Input percentages into array a[n]
4. Set i = 0
5. Repeat Steps 6 to 11 while i < n-1
6. Set min = i
7. Set j = i + 1
8. Repeat Steps 9 to 10 while j < n
9. If a[j] < a[min] then set min = j
10. Increment j = j + 1
11. Swap a[i] and a[min]
12. Increment i = i + 1
13. Display sorted array
14. Display top five percentages
15. Stop

Experiment 3 – Hash Table Algorithm

1. Start
2. Define hash table of fixed size
3. Read client name and phone number
4. Generate hash index using hash function
5. If index is empty, insert record
6. Else collision occurs
7. Apply collision resolution technique:
 - Linear Probing → check next sequential slot
 - Quadratic Probing → check slots using square increments
 - Double Hashing → use second hash function
8. Repeat until empty slot is found
9. Insert record
10. For searching, generate hash index and traverse accordingly
11. If key found display telephone number else display "Record not found"
12. Stop

Experiment 4 – Pinnacle Club using Linked List

1. Start
2. Create linked list node containing PRN, Name and next pointer
3. Initialize head = NULL
4. Add President at beginning
5. Add Secretary at end
6. Add members between President and Secretary
7. Delete President from beginning
8. Delete Secretary from end
9. Delete member using PRN
10. Traverse list and count members
11. Display all members
12. Stop

Experiment 5 – Theatre Ticket Booking using Doubly Linked List

1. Start
2. Create doubly linked list for available seats
3. Initialize all seats as available
4. Display available seats by traversing list
5. To book seat, search seat in available list
6. If found, remove from available list and add to booked list
7. Else display "Seat not available"
8. To cancel booking, search seat in booked list
9. If found, remove from booked list and add back to available list
10. Else display "Seat not booked"
11. Stop

Experiment 6 – Infix to Postfix Conversion

1. Start
2. Read infix expression
3. Initialize empty stack
4. Scan expression from left to right
5. If symbol is operand, add to postfix
6. If symbol is '(', push into stack
7. If symbol is operator, pop higher precedence operators and add to postfix
8. Push current operator into stack
9. If symbol is ')', pop until '(' is found
10. Pop remaining operators from stack
11. Display postfix expression
12. Stop

Experiment 6 – Postfix Evaluation

1. Start
2. Read postfix expression
3. Initialize empty stack
4. Scan postfix expression from left to right
5. If symbol is operand, push into stack
6. If symbol is operator, pop two operands and perform operation
7. Push result back into stack
8. Repeat until expression ends
9. Display final result
10. Stop

Experiment 7 – Pizza Parlor Queue

1. Start
2. Initialize queue with front = rear = -1
3. Read maximum number of orders M
4. To add order, check overflow condition
5. If queue not full, insert order at rear
6. To serve order, check underflow condition
7. If queue not empty, delete order from front
8. To display orders, traverse queue from front to rear
9. Stop

Experiment 8 – Binary Search Tree Operations

1. Start
2. Create BST root node
3. Insert nodes according to BST property
4. For longest path, recursively calculate height
5. For minimum value, traverse leftmost node
6. For mirror tree, swap left and right child recursively
7. For search, compare key and traverse left/right subtree
8. Display inorder traversal
9. Stop

Experiment 9 – Graph using Adjacency List

1. Start
2. Create adjacency list representation of graph
3. Read source city, destination city and cost
4. Add edge between cities
5. Repeat for all flights
6. Display adjacency list
7. Mark all vertices as unvisited
8. Perform DFS traversal
9. Mark visited nodes
10. If all nodes visited, graph is connected
11. Else graph is not connected
12. Stop